CE152: Object-Oriented Programming

Loops

Dr. Sebastian Halder

CSEE

2023

Refresher conditionals

Conditionals

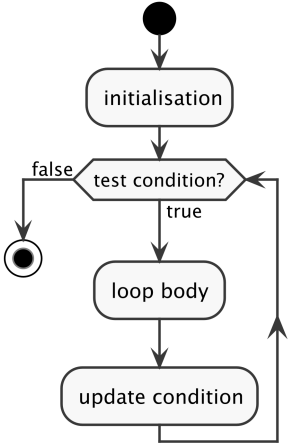
- In the last video we learned how to use conditionals to execute certain sections of code
- We used:
  - `if` : section is executed if the condition is true
  - `else`: follows an `if` and is always executed if the `if` statement is false
  - `else if`: also follows an `if` but is only executed if its own statement is true
- In this video we will learn how to execute a segment of code multiple times using *loops*

Loops

`for` loops

- We can use a `for` loop to repeat a segment of code multiple times
- A for loop has three statements in the *loop header*
  - initialisation
  - test condition
  - update condition
- The *loop body* is executed only if the test condition is true at least once
- The variable we initialise is only accessible in the scope of the *loop body*

`for` loop in UML



for loop in Java

```
for(int i=0; i<10; i++)
{
  System.out.println("i = " + i);
}
```

Combined with if statment

```
for(int i=0; i<10; i++)
{
  if(i>5)
    System.out.println("i = " + i); // We can omit the {}
}
```

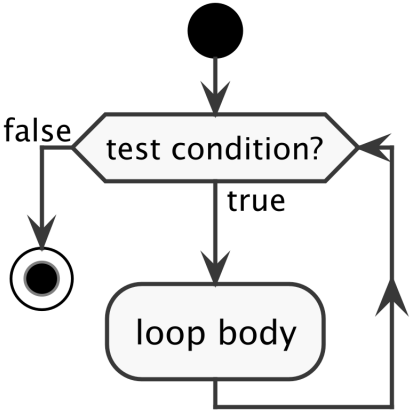
We can use other data types

```
for(double d=0.05; d<0.2; d+=0.01)
{
    System.out.println("d = " + d);
}
```

while loops

- A while loop has only one condition in the header
  - test condition
- We are responsible for creating the variables before hand and updating the variable in the loop body so that it will terminate at some point
- The advantage is now the variable will be accessible outside of the scope of the loop
- Again the body will never be executed if the test condition starts out false
- **If we do not set the correctly condition or update correctly the loop will run forever and the program will appear to be stuck or produce continuous output**

`while` loop in UML



while loop in Java

```
int i = 0;
while (i<5){
  i++; // Again we could omit the {}
}
```

Combining conditions with logical operators

- This also works with `for` loops

```
int x = 5;
int i = 0;

while (i<5 && x < 7){
  i++;
  x++;
}
```

- How many times will the `while` loop run? How many times will it run if we change the `&&` to `||`?

break

- We can also exit loops (for and while) using the keyword *break*

```
while (...){
  ...
  if (...)
    break;
  ...
}
```

- This will exit the loop, *not* execute the rest of the body
- As you see here it is not necessary to put curly brackets around single line condition or loop bodies (in this case the if statement)

continue

- We can jump to the next iteration of the loop (for and while) using the keyword *continue*

```
while (...){
  ...
  if (...)
    continue;
  ...
}
```

- This will *not* exit the loop, *not* execute the rest of the body and continue with the next iteration
- Do not overuse *break* and *continue* as they tend to make code harder to understand

Reading input

- Sometimes we may want to read input from the command line
- For this we use:

```
import java.util.Scanner;
```

- We use it to read from the command line (System.in):

```
Scanner input = new Scanner(System.in);
```

- The above line creates a new Scanner Object in a variable that we named input
- Currently it would read one word at a time but we want to enter multiple words
- We can accomplish this by setting the delimiter we want to use (newline)

```
Scanner input =
  new Scanner(System.in).useDelimiter("\n");
String value = input.next();
int numIterations = Integer.parseInt(value);
```

Live coding

- Write program that reads a number from the command line and runs the for loop as many times

Summary

`while` and `for`

- To execute a segment of code multiple times we can use `while` and `for` loops
- What to use when?
  - Often `for` loops are used when we know exactly how many times the code should run
  - If we do not know because it depends on user input or otherwise unknown conditions we use `while` loops
  - For example if you are waiting for something to be available (e.g. a file) you can run a `while` loop (with pauses!) until you can read the file and then exit the loop with `break`

Thank you and see you in the next video!